Implement depth first search algorithm and breadth first search algorithm , use an undirected graph and develop recursive algorithm for searching all the vertices of a graph or tree data structure.

Code :-

DFS :-

```
graph = {
   'A': ['B','C'],
   'B': ['A','D'],
   'C': ['A','E'],
   'D': ['B'],
   'E': ['C']
}

visited = set()

def dfs(node):
   if node not in visited:
      print(node, end=" ")
      visited.add(node)
      for n in graph[node]:
```

```
        dfs(n)


print("DFS:")
dfs('A')
```

BFS :-

```
from collections import deque


graph = {
   'A': ['B','C'],
   'B': ['A','D'],
   'C': ['A','E'],
   'D': ['B'],
   'E': ['C']
}


def bfs(start):
   visited = set([start])
   q = deque([start])
```

```
    while q:
        node = q.popleft()
        print(node, end=" ")

        for n in graph[node]:
            if n not in visited:
                visited.add(n)
                q.append(n)

print("BFS:")
bfs('A')
```

Theory :- ☑ Aim

To implement DFS (recursive) and BFS algorithms to traverse all vertices of an undirected graph.

☑ Theory

Graph traversal is the process of visiting all the vertices of a graph. Two important traversal techniques are:

- **1. Depth First Search (DFS)**

DFS explores the graph deeply. It starts from a node, visits it, and then recursively visits its adjacent unvisited nodes until no more nodes are left.

- **Uses recursion (stack)**
- **Goes deep first, then backtracks**
- **Suitable for tree/graph traversal, cycle detection**

◇ **2. Breadth First Search (BFS)**

BFS explores the graph level by level. It first visits all neighbors of a node before moving to the next level.

- **Uses queue (FIFO)**
- **Visits nodes level-wise**
- **Useful for shortest path in unweighted graph**

◇ **Undirected Graph**

In an undirected graph, edges are bidirectional (if A is connected to B, then B is also connected to

A).

☑ **Algorithm Steps**

◇ **A) DFS (Recursive)**

1. **Start**
2. **Select a starting node**
3. **Mark the node as visited and print it**
4. **For each adjacent node:**
   - **If not visited, call DFS recursively**
5. **Repeat until all nodes are visited**
6. **Stop**

👉 **To traverse all vertices**

(disconnected graph):

- **Repeat DFS for every unvisited node**

- **B) BFS**

1. **Start**
2. **Select a starting node**
3. **Create a queue and add the starting node**
4. **Mark it as visited**
5. **While queue is not empty:**
    - **Remove node from queue**
    - **Print the node**
    - **Add all**

unvisit

ed

neighb

ors to

queue

and

mark

them

visited

6. **Repeat until**

   queue is

   empty

7. **Stop**